

Listagem do Programa-Fonte

Sistema IoT com ESP32, BMP180, DHT22 e MQ135

Henry Barbosa Pires de Souza
Piero Calabrese Galindo

25 de junho de 2026

Microcontrolador

ESP32 (Espressif Systems)

Sensores

- BMP180
- DHT22
- MQ135

Interface de Programação

Arduino IDE

Linguagem utilizada: C++

Bibliotecas empregadas:

- WiFi.h
- WebServer.h
- Wire.h
- Adafruit_BMP085.h
- MQUnifiedsensor.h
- DHT.h

Programa-Fonte

```
1
2 /*
3  * ESP32 - Dashboard Web
4  * Sensores: DHT22, BMP180, MQ135
5  * Henry Barbosa e Piero Calabrese
6  *
7  * - Scanner I2C para diagnóstico
8  * - Tratamento de R0 = inf no MQ135
9  * - Classes de qualidade do ar
10 * - Fallback seguro para sensores
11 */
12
13 #include <WiFi.h>
14 #include <WebServer.h>
15 #include <DHT.h>
16 #include <Wire.h>
17 #include <Adafruit_BMP085.h>
18 #include <MQUnifiedsensor.h>
19
20 // ===== CONFIGURA ES DE REDE =====
21 const char* ssid = "";
22 const char* password = "";
23
24 IPAddress local_IP(192, 168, 0, 50);
25 IPAddress gateway(192, 168, 0, 1);
26 IPAddress subnet(255, 255, 255, 0);
27 IPAddress primaryDNS(8, 8, 8, 8);
28 IPAddress secondaryDNS(8, 8, 4, 4);
29
30 // ===== PINAGEM DOS SENSORES =====
31 #define DHT22_PIN 27
32 #define DHT_TYPE DHT22
33 #define BMP_SDA 25
34 #define BMP_SCL 26
35 #define MQ135_PIN 34
36
37 // ===== INICIALIZA O DOS SENSORES =====
38 DHT dht(DHT22_PIN, DHT_TYPE);
39 Adafruit_BMP085 bmp;
40
41 // Configura o MQUnifiedsensor
42 #define Board ("ESP-32")
43 #define Voltage_Resolution (3.3)
44 #define ADC_Bit_Resolution (12)
45 #define RatioMQ135CleanAir (3.6)
46 MQUnifiedsensor MQ135(Board, Voltage_Resolution, ADC_Bit_Resolution
47 , MQ135_PIN, "MQ-135");
48
49 WebServer server(80);
```

```

49
50 // ===== VARI VEIS =====
51 float temperatura = NAN;
52 float umidade     = NAN;
53 float pressao     = NAN;
54 float altitude    = NAN;
55 float qualidadeAr = NAN;
56 float rZero       = NAN;
57
58 unsigned long lastRead = 0;
59 const unsigned long interval = 2000;
60
61 bool bmpOk = false;
62
63 // ===== CLASSIFICA O DA QUALIDADE DO AR =====
64 String classificarQualidade(float ppm) {
65     if (isnan(ppm) || ppm <= 0) return "Sem dado";
66     if (ppm < 400) return "Excelente";
67     if (ppm < 600) return "Boa";
68     if (ppm < 1000) return "Moderada";
69     if (ppm < 1500) return "Alerta";
70     if (ppm < 2000) return "Cr tica";
71     return "Perigosa";
72 }
73
74 // ===== LEITURA DOS SENSORES =====
75 void lerSensores() {
76     // --- DHT22 ---
77     float t = dht.readTemperature();
78     float h = dht.readHumidity();
79     if (!isnan(t) && !isnan(h)) {
80         temperatura = t;
81         umidade = h;
82     }
83
84     // --- BMP180 ---
85     if (bmpOk) {
86         pressao = bmp.readPressure() / 100.0F;
87         altitude = bmp.readAltitude(1013.25);
88     } else {
89         pressao = 925.0; // valor fixo de fallback
90         altitude = NAN; // altitude continua sem dado
91     }
92
93     // --- MQ135 ---
94     MQ135.setA(110);
95     MQ135.setB(-0.63);
96     MQ135.update();
97
98     float leitura = MQ135.readSensor();
99     if (!isinf(leitura) && !isnan(leitura) && leitura > 0) {

```

```

100     qualidadeAr = leitura;
101 }
102 rZero = MQ135.getR0();
103 }
104
105 // ===== P GINA HTML =====
106 String gerarPagina() {
107     String pagina = R"rawliteral(
108 <!DOCTYPE html>
109 <html lang="pt-br">
110 <head>
111 <meta charset="utf-8">
112 <title>SCADA Industrial</title>
113 <script src="https://cdn.jsdelivrivr.net/npm/chart.js"></script>
114 <style>
115 body {
116     margin: 0;
117     font-family: Arial;
118     background: #0b1220;
119     color: #d1d5db;
120 }
121 .header {
122     background: #0f172a;
123     padding: 12px;
124     text-align: center;
125     border-bottom: 2px solid #1f2937;
126 }
127 .header h1 {
128     margin: 0;
129     color: #38bdf8;
130     font-size: 18px;
131 }
132 .grid {
133     display: grid;
134     grid-template-columns: repeat(4, 1fr);
135     gap: 12px;
136     padding: 15px;
137 }
138 .card {
139     background: #111827;
140     border: 1px solid #1f2937;
141     border-radius: 10px;
142     padding: 12px;
143     text-align: center;
144 }
145 .label {
146     font-size: 12px;
147     color: #9ca3af;
148 }
149 .value {
150     font-size: 26px;

```

```

151     font-weight: bold;
152     margin-top: 8px;
153 }
154 .ok { color: #22c55e; }
155 .warn { color: #facc15; }
156 .bad { color: #ef4444; }
157 .gaugeBox {
158     display: grid;
159     grid-template-columns: repeat(4, 1fr);
160     gap: 12px;
161     padding: 15px;
162 }
163 .gaugeCard {
164     background: #111827;
165     border: 1px solid #1f2937;
166     border-radius: 10px;
167     padding: 12px;
168     text-align: center;
169 }
170 .gauge {
171     width: 140px;
172     height: 70px;
173     margin: 10px auto;
174     border-radius: 140px 140px 0 0;
175     background: #1f2937;
176     position: relative;
177     overflow: hidden;
178 }
179 .fill {
180     position: absolute;
181     bottom: 0;
182     width: 100%;
183     height: 50%;
184     transition: 0.4s;
185 }
186 .chartBox {
187     margin: 15px;
188     background: #111827;
189     border: 1px solid #1f2937;
190     padding: 10px;
191     border-radius: 10px;
192 }
193 .footer {
194     background: #0f172a;
195     padding: 10px;
196     font-size: 12px;
197     border-top: 1px solid #1f2937;
198     display: flex;
199     justify-content: space-between;
200 }
201 </style>

```

```

202 </head>
203 <body>
204 <div class="header">
205   <h1>SISTEMA SCADA - MONITORAMENTO DE CONDIÇÕES FÍSICAS </h1>
206 </div>
207 <div class="grid">
208   <div class="card">
209     <div class="label">TEMPERATURA </div>
210     <div id="temp" class="value">--</div>
211   </div>
212   <div class="card">
213     <div class="label">UMIDADE </div>
214     <div id="hum" class="value">--</div>
215   </div>
216   <div class="card">
217     <div class="label">PRESSÃO </div>
218     <div id="press" class="value">--</div>
219   </div>
220   <div class="card">
221     <div class="label">QUALIDADE DO AR </div>
222     <div id="gasText" class="value ok">BOA </div>
223   </div>
224 </div>
225 <div class="gaugeBox">
226   <div class="gaugeCard">
227     <div class="label">TEMPERATURA </div>
228     <div class="gauge"><div id="gTemp" class="fill" style="height
229       :50%;background:#ef4444;"></div></div>
230     <div id="tVal" class="value">--</div>
231   </div>
232   <div class="gaugeCard">
233     <div class="label">UMIDADE </div>
234     <div class="gauge"><div id="gHum" class="fill" style="height
235       :50%;background:#22c55e;"></div></div>
236     <div id="hVal" class="value">--</div>
237   </div>
238   <div class="gaugeCard">
239     <div class="label">PRESSÃO </div>
240     <div class="gauge"><div id="gPress" class="fill" style="height
241       :50%;background:#3b82f6;"></div></div>
242     <div id="pVal" class="value">--</div>
243   </div>
244   <div class="gaugeCard">
245     <div class="label">QUALIDADE DO AR </div>
246     <div class="gauge"><div id="gGas" class="fill" style="height
247       :50%;background:#22c55e;"></div></div>
248     <div id="gVal" class="value">--</div>
249   </div>
250 </div>
251 <div class="chartBox">
252   <div class="label">TENDÊNCIA DO SISTEMA </div>

```

```

249     <canvas id="chart"></canvas>
250 </div>
251 <div class="footer">
252     <div>STATUS: <span style="color:#22c55e">ONLINE</span></div>
253     <div>Desenvolvido por Henry Barbosa e Piero Calabrese</div>
254 </div>
255 <script>
256 function gasStatus(v) {
257     if (v < 400) return { txt: "EXCELENTE", color: "#22c55e" };
258     if (v < 600) return { txt: "BOA", color: "#76ff03" };
259     if (v < 1000) return { txt: "MODERADA", color: "#facc15" };
260     if (v < 1500) return { txt: "ALERTA", color: "#ff9800" };
261     if (v < 2000) return { txt: "CR TICA", color: "#ef4444" };
262     return { txt: "PERIGOSA", color: "#d50000" };
263 }
264 const ctx = document.getElementById("chart");
265 const chart = new Chart(ctx, {
266     type: "line",
267     data: {
268         labels: [],
269         datasets: [
270             { label: "Temperatura", data: [], borderColor: "red",
271               yAxisID: "yTemp", tension: 0.3 },
272             { label: "Umidade", data: [], borderColor: "cyan",
273               yAxisID: "yHum", tension: 0.3 },
274             { label: "Press o", data: [], borderColor: "lime",
275               yAxisID: "yPress", tension: 0.3 },
276             { label: "Qualidade do Ar", data: [], borderColor: "orange",
277               yAxisID: "yGas", tension: 0.3 }
278         ]
279     },
280     options: {
281         animation: false,
282         plugins: { legend: { labels: { color: "#d1d5db" } } },
283         scales: {
284             x: { ticks: { color: "#9ca3af" } },
285             yTemp: { type: "linear", position: "left", min: 10, max: 40,
286                   ticks: { stepSize: 5, color: "#9ca3af" } },
287             yHum: { type: "linear", position: "right", min: 0, max: 100,
288                   ticks: { stepSize: 20, color: "#9ca3af" }, grid: {
289                     drawOnChartArea: false } },
290             yPress: { type: "linear", position: "right", min: 900, max: 1025,
291                     ticks: { stepSize: 25, color: "#9ca3af" }, grid: {
292                     drawOnChartArea: false } },
293             yGas: { type: "linear", position: "left", min: 0, max: 3000,
294                   ticks: { stepSize: 500, color: "#9ca3af" }, grid: {
295                     drawOnChartArea: false } }
296         }
297     }
298 });
299 async function fetchData() {

```

```

289 try {
290     const resp = await fetch("/data");
291     const d = await resp.json();
292     const t = new Date().toLocaleTimeString();
293     const temp = parseFloat(d.temperatura) || 0;
294     const hum = parseFloat(d.umidade) || 0;
295     const press = parseFloat(d.pressao) || 925;
296     const gas = parseFloat(d.qualidadeAr) || 0;
297     const st = gasStatus(gas);
298     document.getElementById("temp").innerText = temp.toFixed(1) + "
        C ";
299     document.getElementById("hum").innerText = hum.toFixed(1) + "
        %";
300     document.getElementById("press").innerText = press.toFixed(1) + "
        hPa";
301     const ge = document.getElementById("gasText");
302     ge.innerText = st.txt;
303     ge.style.color = st.color;
304     let tPct = Math.min(100, Math.max(0, ((temp - 10) / 30) * 100))
        ;
305     document.getElementById("gTemp").style.height = tPct + "%";
306     document.getElementById("gTemp").style.background = "#ef4444";
307     document.getElementById("tVal").innerText = temp.toFixed(1) + "
        C ";
308     let hPct = Math.min(100, Math.max(0, hum));
309     document.getElementById("gHum").style.height = hPct + "%";
310     document.getElementById("gHum").style.background = "#22c55e";
311     document.getElementById("hVal").innerText = hum.toFixed(1) + "%
        ";
312     let pPct = Math.min(100, Math.max(0, ((press - 900) / 125) *
        100));
313     document.getElementById("gPress").style.height = pPct + "%";
314     document.getElementById("gPress").style.background = "#3b82f6";
315     document.getElementById("pVal").innerText = press.toFixed(1);
316     let gPct = Math.min(100, (gas / 3000) * 100);
317     document.getElementById("gGas").style.height = gPct + "%";
318     document.getElementById("gGas").style.background = st.color;
319     document.getElementById("gVal").innerText = Math.floor(gas);
320     chart.data.labels.push(t);
321     chart.data.datasets[0].data.push(temp);
322     chart.data.datasets[1].data.push(hum);
323     chart.data.datasets[2].data.push(press);
324     chart.data.datasets[3].data.push(gas);
325     if (chart.data.labels.length > 30) {
326         chart.data.labels.shift();
327         chart.data.datasets.forEach(ds => ds.data.shift());
328     }
329     chart.update();
330 } catch(e) {
331     console.error("Erro", e);
332 }

```



```

333 }
334 setInterval(fetchData, 2000);
335 fetchData();
336 </script>
337 </body>
338 </html>
339 )rawliteral";
340
341     return pagina;
342 }
343 // ===== ROTAS =====
344 void handleRoot() {
345     server.send(200, "text/html; charset=utf-8", gerarPagina());
346 }
347
348 void handleData() {
349     String json = "{";
350     json += "\"temperatura\":" + String(isnan(temperatura) ? 0.0 :
        temperatura, 1) + ",";
351     json += "\"umidade\":" + String(isnan(umidade) ? 0.0 : umidade,
        1) + ",";
352     json += "\"pressao\":" + String(isnan(pressao) ? 0.0 : pressao,
        1) + ",";
353     json += "\"altitude\":" + String(isnan(altitude) ? 0.0 : altitude
        , 1) + ",";
354     json += "\"qualidadeAr\":" + String(isnan(qualidadeAr) ? 0 :
        qualidadeAr, 0) + ",";
355     json += "\"classificacao\":" + classificarQualidade(qualidadeAr
        ) + "\"";
356     json += "}";
357     server.send(200, "application/json", json);
358 }
359
360 void handleNotFound() {
361     server.send(404, "text/plain", "404 - Pagina nao encontrada");
362 }
363
364 // ===== SETUP =====
365 void setup() {
366     Serial.begin(115200);
367     delay(500);
368     Serial.println("\n== Iniciando Estacao Ambiental ESP32 ==");
369
370     // I2C para BMP180
371     Wire.begin(BMP_SDA, BMP_SCL);
372
373     // Scanner I2C para diagnostico
374     Serial.println("Escaneando I2C...");
375     byte count = 0;
376     for (byte addr = 1; addr < 127; addr++) {
377         Wire.beginTransmission(addr);

```

```

378     if (Wire.endTransmission() == 0) {
379         Serial.print("Dispositivo I2C encontrado em 0x");
380         Serial.println(addr, HEX);
381         count++;
382     }
383     delay(5);
384 }
385 if (count == 0) Serial.println("Nenhum dispositivo I2C encontrado
    !");
386
387 // DHT22
388 dht.begin();
389
390 // BMP180
391 if (bmp.begin()) {
392     Serial.println("BMP180 OK.");
393     bmpOk = true;
394 } else {
395     Serial.println("BMP180 NAO ENCONTRADO!");
396     bmpOk = false;
397 }
398
399 // MQ135 - Configura o
400 MQ135.setRegressionMethod(1);
401 MQ135.init();
402 MQ135.setR0(10.0);
403
404 MQ135.setA(110.0);
405 MQ135.setB(-0.63);
406
407 Serial.print("Calibrando MQ135. Aguarde...");
408 float calcR0 = 0;
409 int amostrasValidas = 0;
410 for (int i = 1; i <= 10; i++) {
411     MQ135.update();
412     float leitura = MQ135.calibrate(RatioMQ135CleanAir);
413     if (!isinf(leitura) && !isnan(leitura) && leitura > 0) {
414         calcR0 += leitura;
415         amostrasValidas++;
416     }
417     Serial.print(".");
418     delay(500);
419 }
420
421 if (amostrasValidas > 0) {
422     MQ135.setR0(calcR0 / amostrasValidas);
423     Serial.println("\nMQ135 calibrado! R0 = " + String(MQ135.getR0
        ()));
424 } else {
425     MQ135.setR0(10.0);

```

```

426     Serial.println("\nFalha na calibracao MQ135 (R0=inf), usando R0
         =10.0 como fallback");
427 }
428
429 // WiFi - IP fixo
430 if (!WiFi.config(local_IP, gateway, subnet, primaryDNS,
         secondaryDNS)) {
431     Serial.println("Falha ao configurar IP fixo!");
432 }
433
434 WiFi.begin(ssid, password);
435 Serial.print("Conectando WiFi...");
436
437 int tentativas = 0;
438 while (WiFi.status() != WL_CONNECTED && tentativas < 40) {
439     delay(500);
440     Serial.print(".");
441     tentativas++;
442 }
443
444 if (WiFi.status() == WL_CONNECTED) {
445     Serial.println("\nWiFi OK!");
446     Serial.print("IP fixo: http://");
447     Serial.println(WiFi.localIP());
448 } else {
449     Serial.println("\nFalha WiFi. Reiniciando...");
450     ESP.restart();
451 }
452
453 // Servidor
454 server.on("/", handleRoot);
455 server.on("/data", handleData);
456 server.onNotFound(handleNotFound);
457 server.begin();
458 Serial.println("Servidor rodando na porta 80");
459 }
460
461 // ===== LOOP =====
462 void loop() {
463     server.handleClient();
464
465     unsigned long now = millis();
466     if (now - lastRead >= interval) {
467         lastRead = now;
468         lerSensores();
469
470         Serial.print("Temp: ");
471         if (isnan(temperatura)) Serial.print("--");
472         else { Serial.print(temperatura, 1); }
473         Serial.print(" C | Umi: ");
474         if (isnan(umidade)) Serial.print("--");

```

```
475     else { Serial.print(umidade, 1); }
476     Serial.print("% | Press: ");
477     if (isnan(pressao)) Serial.print("--");
478     else { Serial.print(pressao, 1); }
479     Serial.print("hPa | CO2: ");
480     if (isnan(qualidadeAr)) Serial.print("--");
481     else { Serial.print(qualidadeAr, 0); }
482     Serial.print("ppm | ");
483     Serial.println(classificarQualidade(qualidadeAr));
484 }
485 }
```